

UNIT – 1
ASSESSMENT – 1
SOLUTIONS

1. Represent the state as:

(Amount in 4-gallon jug, Amount in 3-gallon jug)

Initially,

(0,0)

Step	Action	State (4-gallon,3-gallon)
1	Fill 3-gallon jug	(0,3)
2	Pour into 4-gallon jug	(3,0)
3	Fill 3-gallon jug again	(3,3)
4	Pour into 4-gallon jug until full	(4,2)
5	Empty the 4-gallon jug	(0,2)
6	Pour remaining 2 gallons into 4-gallon jug	(2,0)

Final state:

(2,0)

Hence,

Exactly 2 gallons are obtained in the 4-gallon jug.

State Space Representation:

Start

(0,0)



Fill 3-gallon



(0,3)



Pour to 4-gallon



(3,0)



Fill 3-gallon



(3,3)



Pour to 4-gallon



(4,2)



Empty 4-gallon



(0,2)



Pour into 4-gallon

↓

(2,0)

Goal Achieved

2. Percepts are the information received by the rover through its sensors.

The Mars Rover collects the following percepts:

- Images captured by cameras
- Surface temperature
- Atmospheric pressure
- Soil composition
- Rock composition
- Terrain obstacles
- Distance to nearby objects
- Position and direction (GPS-like localization and navigation systems)
- Battery level
- Wheel movement and speed
- Weather conditions such as dust storms and wind

These percepts help the rover understand its surroundings and make intelligent decisions.

2. Characterize the Operating Environment

The operating environment of the Mars Rover can be described as follows:

Environment Property	Description
Observable	Partially Observable
Agents	Single Agent
Deterministic	Mostly Stochastic (uncertain due to weather and terrain)
Episodic / Sequential	Sequential
Static / Dynamic	Dynamic
Discrete / Continuous	Continuous
Known / Unknown	Partially Known

Explanation

- The rover cannot observe the entire Martian surface.
- Weather conditions may change unexpectedly.
- Every action affects future actions.
- The rover continuously receives new sensor data.
- The environment changes with time.

3. What actions can the rover perform?

The Mars Rover can perform the following actions:

- Move forward
- Move backward
- Turn left
- Turn right
- Stop
- Capture images
- Drill rocks
- Collect soil samples
- Analyze rocks and soil
- Measure temperature
- Measure atmospheric conditions
- Avoid obstacles
- Send collected data to Earth
- Recharge using solar panels (when available)

4. How is the performance evaluated?

The performance of the Mars Rover is evaluated based on:

- Successful navigation
- Number of scientific samples collected
- Accuracy of data analysis
- Distance covered
- Number of obstacles avoided
- Battery efficiency

- Communication success with Earth
- Mission completion
- Equipment reliability
- Safety of the rover

A rover with higher scientific output and efficient resource usage is considered more successful.

5. Suitable Agent Architecture

Learning Agent Architecture

A **Learning Agent** is the most suitable architecture for the Mars Rover.

Reason

A learning agent can:

- Learn from previous experiences.
- Improve navigation over time.
- Adapt to unknown terrain.
- Handle unexpected situations.
- Optimize energy usage.
- Improve obstacle avoidance.
- Make better decisions without constant human control.

Since Mars is an unknown and constantly changing environment, a learning agent is ideal.

Conclusion

The Mars Rover is an intelligent autonomous agent that continuously perceives its environment, makes decisions, learns from experience, and performs scientific exploration with minimal human intervention.

3. The 8-Queens Problem is one of the most popular Artificial Intelligence search problems.

The goal is to place eight queens on a chessboard so that no queen attacks another queen.

The problem is solved using **State Space Search** and **Backtracking Search**, where invalid placements are rejected and the algorithm tries another possible position.

Problem Formulation

1. Initial State

An empty **8 × 8 chessboard** with no queens placed.

.....

.....

.....

.....

.....

.....

.....

.....

2. State Space

Each state represents a partially filled chessboard.

3. Operators

The allowed operation is:

- Place one queen in a safe position in the next row.

If no safe position exists, backtrack to the previous row.

4. Goal State

A board containing **8 queens** such that:

- No two queens share the same row.
- No two queens share the same column.
- No two queens share the same diagonal.

Example of one valid solution:

Q

. . . . Q . . .

. Q

. Q . .

. . Q

. Q .

. Q

. . . Q

Search Strategy

The **Backtracking Search Algorithm** is used.

Working

1. Place a queen in the first row.
2. Move to the next row.

3. Check whether the new queen is safe.
4. If safe, continue.
5. Otherwise, try another column.
6. If no column is safe, return to the previous row.
7. Continue until all eight queens are placed.

Advantages

- Efficient compared to checking every arrangement.
- Eliminates invalid states early.
- Guarantees finding a solution if one exists.

4. Type of Problem-Solving Agent

The most suitable agent for this problem is a Goal-Based Agent.

Reason

A Goal-Based Agent makes decisions based on achieving a specific goal.

In the OLA application:

- Initial State: Customer opens the OLA app.
- Goal: Reach the desired destination.
- Possible Actions: Select a cab type, confirm booking, and travel.
- Decision Making: The agent compares different cab options and selects the one that best satisfies the customer's requirements such as price, comfort, availability, and estimated arrival time.

Thus, a Goal-Based Agent is appropriate because it plans actions to achieve the user's goal of reaching the destination.

Working of the Goal-Based Agent

1. Customer enters the pickup location.
2. Customer enters the destination.
3. The application calculates the route.
4. Available cab types are displayed.
5. The customer selects a preferred cab.
6. The booking is confirmed.
7. The driver is assigned.
8. The customer travels to the destination.
9. The trip ends successfully.

PSEUDOCODE:

BEGIN

Input Pickup_Location

Input Destination

Display Available_Cabs

Input Selected_Cab

IF Selected_Cab is Available THEN

 Book Cab

 Assign Driver

Display Driver Details
Start Trip
Reach Destination
Display "Trip Completed"

ELSE

Display "Selected Cab Not Available"

ENDIF

END

FLOW OF THE PROCESS:

Start



Enter Pickup Location



Enter Destination



Display Available Cabs



Select Cab Type



Cab Available?



Yes ----- No



Book Cab Display Not Available

|

Assign Driver

|

Start Trip

|

Reach Destination

|

Trip Completed

|

End

5. Uniform Cost Search (UCS) is an uninformed search algorithm that expands the node with the lowest cumulative path cost first. It guarantees the optimal (least-cost) path when all edge costs are non-negative.

Unlike Breadth-First Search (BFS), UCS considers the actual cost of traveling rather than simply the number of edges.

Step-by-Step UCS Traversal

Initial State

Frontier = {(S,0)}

Expand S

Neighbors:

- A (Cost = 1)
- G (Cost = 12)

Frontier:

(A,1), (G,12)

Expand A (lowest cost)

Neighbors:

- $B = 1 + 3 = 4$
- $C = 1 + 1 = 2$

Frontier:

(C,2), (B,4), (G,12)

Expand C

Neighbor:

- $D = 2 + 1 = 3$

Frontier:

(D,3), (B,4), (G,12)

Expand D

Neighbor:

- $G = 3 + 3 = 6$

Frontier:

(B,4), (G,6), (G,12)

Expand B

Neighbor:

- $G = 4 + 3 = 7$

Frontier:

$(G,6), (G,7), (G,12)$

Expand G

Goal reached.

Least-Cost Path

$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$

Total Cost

$S \rightarrow A = 1$

$A \rightarrow C = 1$

$C \rightarrow D = 1$

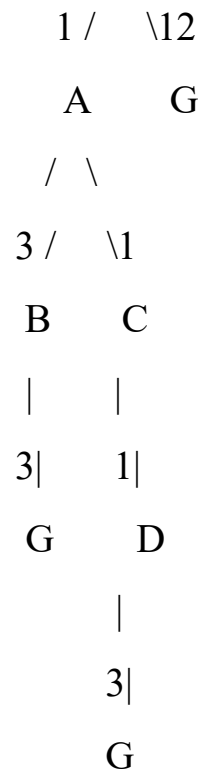
$D \rightarrow G = 3$

Total Cost = $1 + 1 + 1 + 3 = 6$

Search Tree

S

/ \



Conclusion

The least-cost path found by Uniform Cost Search is:

$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$

with a minimum cost of

6